2. Trace the actual publishers:
   `TimelineController#select`
   `TripNavController#show`
3. Trace the actual subscriber:
   `MapController`
4. Show how listener registration, event dispatch and notification handling work using the real source.
5. Check whether the producers directly call `MapController` methods or instead notify through the browser event mechanism.
6. Then challenge the classification:
   the producers know the map DOM element
   they know the exact event names
   there is effectively one receiver
   browser `EventTarget` supplies the observer infrastructure
7. Decide whether this is sufficiently substantive to call Observer, or whether "targeted DOM event communication" is the more accurate description.
8. Give a final decision:
   CONFIRMED
   REJECTED
   UNCERTAIN

If confirmed, keep all three `map:*` events inside one instance.
For Trip-Scoped Turbo Refresh Notifications:

1. Use the application evidence already found:
   `turbo_stream_from @trip`
   `broadcasts_refreshes_to :trip`
   `broadcast_refresh_to(trip)`
2. Use the checked Turbo Rails 2.0.23 framework contract to identify publisher, subscription and delivery roles.
3. Decide whether this qualifies as a framework-mediated Observer collaboration under the same reasoning used for the already confirmed `Outbound Mail Delivery Logging` Observer.
4. Compare the two directly:
   what application code defines
   what framework code supplies
   whether a concrete observer/subscriber exists

how registration/subscription happens
how notification is delivered

5. Explain why the Turbo collaboration should remain uncertain if its evidence is materially weaker than the mail Observer.
6. If it is confirmed, count it once across both geocoding and inbox workflows.
7. If it is rejected, explain why the framework usage is not sufficient for the assignment's pattern threshold.

At the end give:

1. Final decision table for both candidates.
2. Participant-role mapping for any confirmed Observer instance.
3. Stable instance name.
4. Source evidence to preserve.
5. Counting decision.
6. Any remaining runtime questions kept separate from pattern classification.

Do not search for new patterns in this prompt.

Worked for 1m 7s

DaPlanStan — Frontend Observer Verification

Candidate Review 06 · Original source ZIP · Static source and framework-contract verification

1. Final decisions

Itinerary Timeline–Map Event Coordination Proposed GoF Observer	REJECTED
Trip-Scoped Turbo Refresh Notifications Framework-mediated Observer	CONFIRMED

The distinction is based on what the collaborations actually do, not simply whether they use events.

The timeline–map implementation sends commands to a specifically identified map element through browser events. The Turbo implementation establishes a genuine trip-scoped publication/subscription relationship through the framework's documented contract.

The Turbo confirmation is for a **framework-mediated Observer instance**, not an independently implemented application-owned GoF Observer framework. Runtime operation and independent